BSCS FINAL PROJECT

Agile Software Design Specification

PLAYERSZWZ

AI-Powered Sports Matchmaking & Arena Booking Application



Product Owner

Mr. Asif Farooq

Presented by:

Group ID: S26CS024

Student Reg#	Student Name
L1F24PACS0021	Waleed Abid
L1F21BSCS0951	Afaq Ul Islam
L1F22BSCS1053	Hassan Ahmed

Faculty of Information Technology & Computer Science

University of Central Punjab

Agile Software Design Specification

SDS Phase II

Version 2.0

Product Owner

Mr. Asif Farooq

Scrum Master

Afaq Ul Islam

Group S26CS024

Team

Member Name	Roles
Waleed Abid	Mobile app architecture, backend integration and project coordination
Afaq Ul Islam	Scrum Master, frontend implementation, maps/location services and UI flows
Hassan Ahmed	AI recommendation module, testing, documentation and QA

Table of Contents

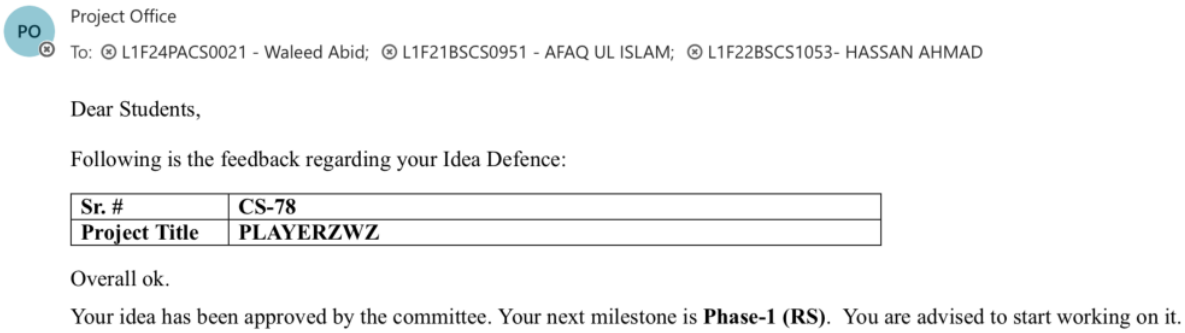
Table of Contents i
Revision History & Sprint Log ii
Previous Phases Feedback ii
Abstract iii
1. System Architecture 1
1.1 High-Level Architecture 1
1.2 Architecture Diagram 2
1.3 Component Overview 3
2. Mapping Design to User Stories 4
3. Detailed System Design 5
3.1 Module Decomposition 5
3.2 Data Flow Description 6
4. UML Diagrams 7
4.1 Use Case Diagram 7
4.2 Class Diagram 8
4.3 Sequence Diagrams 9
5. Database Design (ERD) 10
5.1 ER Diagram 10
5.2 Schema Tables 11
6. API Design 14
7. UI/UX Design (Prototypes) 15
7.1 Navigation Flow 18
7.2 Screens and Prototype Notes 19
7.3 Wireframes Rules 19
8. Sprint-wise Design Evolution 20
9. Test Design 21
10. Design Quality Attributes 22
11. Deployment Considerations 23
12. Future Enhancements 24
13. Revised Project Plan 25
14. References 27
Appendix A: Glossary 28
Appendix B: IV & V Report 29

Revision History & Sprint Log

Version	Sprint/Stage	Date	Changes	Owner
2.0	Design Initiation	08 May 2026	Phase-I SRS reviewed, design scope and SDS structure finalized as per designing for Phase-II following the template.	Afaq Ul Islam
2.1	Architecture & Data Design	18 May 2026	Client-server layered architecture, modules, data flows, ERD and schema were developed.	Waleed Abid
2.2	UML, API & Test Design	02 June 2026	Class diagram, Sequence diagrams, API contracts, UI flow, test design and IV&V log are added.	Hassan Ahmed

Previous Phases Feedback

Idea Defense Feedback (Screenshot)



PO Project Office

To: L1F24PACS0021 - Waleed Abid; L1F21BSCS0951 - AFAQ UL ISLAM; L1F22BSCS1053- HASSAN AHMAD

Dear Students,

Following is the feedback regarding your Idea Defence:

Sr. #	**CS-78**
Project Title	**PLAYERZWZ**

Overall ok.

Your idea has been approved by the committee. Your next milestone is **Phase-1 (RS)**. You are advised to start working on it.

Abstract

PLAYERSZWZ is an AI-powered sports matchmaking and arena booking mobile application that helps streamline the process of arranging casual sports matches. The product vision, stakeholders, epics, user stories, functional requirements, non-functional requirements and risks were defined in the Phase-I requirements specification. In this Phase-II Agile Software Design Specification, all the requirements are turned into a full software design solution. The system is proposed as a layered client-server system comprising of a Flutter mobile app, Firebase Authentication, Flask REST API, Cloud Fire store, Google Maps/Places integration, Firebase Cloud Messaging and a recommendation service. The design enables secure account administration, player profile preferences, discovery of nearby players, creating of games, join requests, booking of the arenas, recommendations, rating/reporting, notification system and admin moderation. This document describes the architectural design, component roles, user story mapping, module decomposition, data flows, UML diagrams, logical database design, API contracts, UI/UX navigation flow, and the evolution of the design over the sprint as well as test design, quality attributes, deployment plan, and future enhancements and an IV&V log. This forms a useful design baseline to implement, test and demonstrate by the development team in the next stage of the final.

1. System Architecture

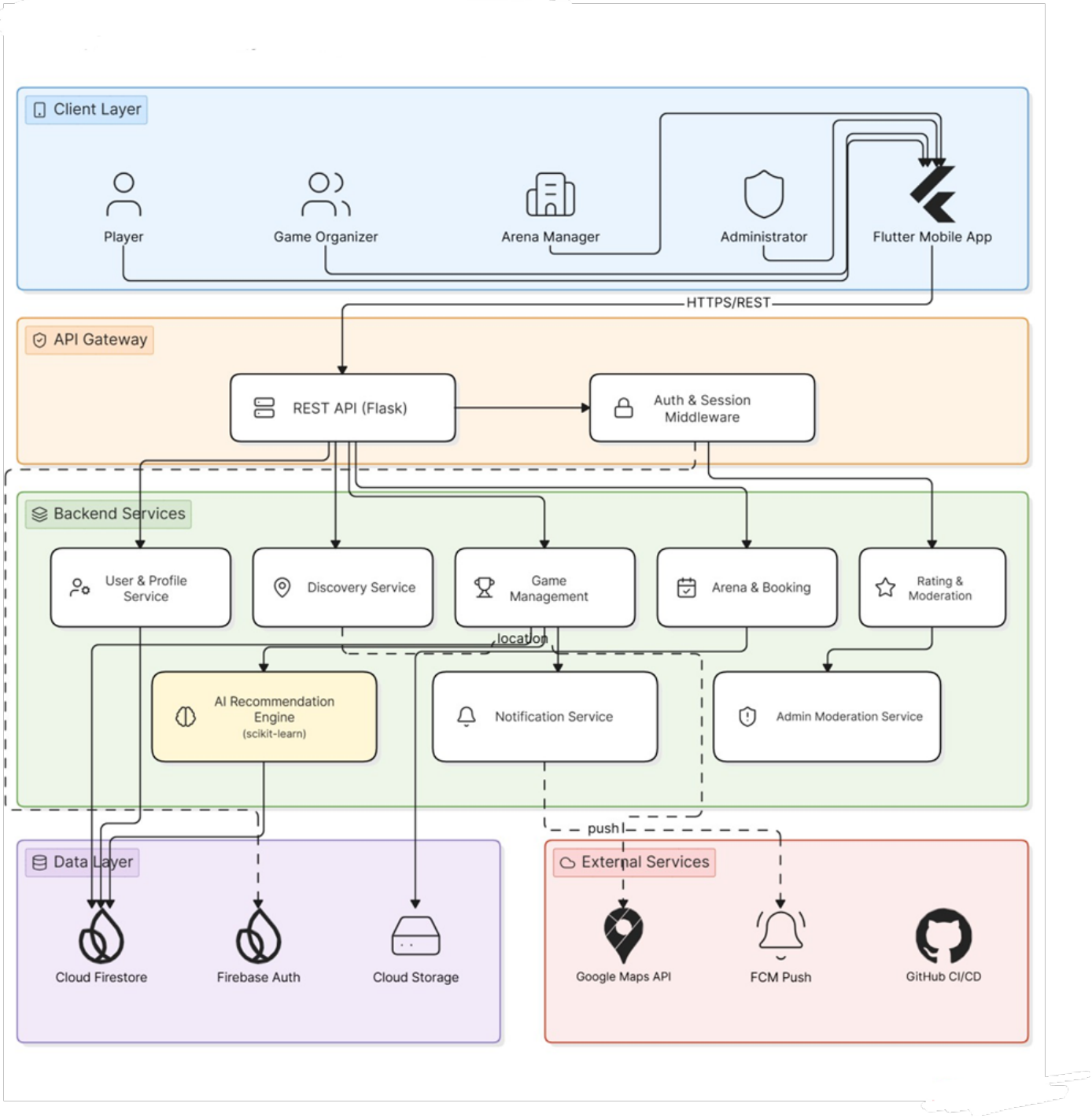
1.1 High-Level Architecture

PLAYERSZWZ is a multi-layered client-server application. The mobile layer includes the Flutter application that players use to play the game, and the game organizers and arena managers use to organize the games. The service layer has REST API controllers, validation rules, recommendation logic and notification triggers. The data layer includes authentication, profile, game, arena, booking, rating and notification data in the Firebase services. External integrations include Firebase Authentication, Cloud Fire store, Google Maps/Places API and Firebase Cloud Messaging.

This is chosen due to the separation of UI logic from business logic and data storage. It also allows the AI recommendation module to be easily replaced: the project can begin by using a purely rule-based ranking system and then later improve the ranking based on history, reliability and availability signals without redesigning the entire mobile application.

Presentation layer	Flutter screens, local form validation, map/list view and navigation.
API layer	Flask controllers, authorization, request validation, status rules and response formatting.
Domain service layer	Game management, booking management, recommendation ranking, feedback dealing and notification scheduling.
Data layer	Firebase Authentication and Cloud Fire store collections as logical database.
External Layer	The layer of external services includes Google Maps/Places (geolocation) and Firebase Cloud Messaging (push notifications).

1.2 Architecture Diagram



1.3 Component Overview

Component	Responsibility
Flutter Mobile App	Offers user register, log in, profile management, map, discovery, game creation, join requests, booking, notification and feedback screens.
Firebase Authentication	Handles email / password sign up, login and safe session tokens.
Flask REST API	Makes Api calls, validates requests, runs role checks and manages reads/writes.
Cloud Fire store	Uses, profiles, games, participants, arenas, bookings, ratings, notifications and admin actions.
Recommendation Service	Ranks games and players using sport preference, distance, availability, past activity and reliability score.
Google Maps/Places API	Displays maps, geocodes, calculates distances & aids in finding an arena/location.
Firebase Cloud Messaging	Push notifications regarding join requests, approval, booking status, reminders and cancellations.
Notification Service	Generates and sends FCM payloads for key events and logs in-app notifications.

2. Mapping Design to User Stories

In the design to follow, each of the Phase-I user stories are mapped to a particular module, database area, API/service contract, and the UI screen that would be presented to the user. This mapping can be used to test the Phase-II design and to ensure the implementation is within the approved requirements.

Story ID	Mapped Design Component	Design Description
US-01	A set of data structures used to identify user profiles and retrieve them from the authentication module.	Create user and profile record (register account with email, password, city and favorite sport).
US-02	Authentication Module, Session Handler	Securely log in and out by interacting with the Firebase token and API authorization middleware.
US-03	Profile & Preferences Module	Adjust the sports, bio, availability, radius and location settings associated with discovery and recommendation.
US-04	There is a Discovery Module available on the nearby side which allows for maps to be integrated.	Ask local players around you to play the game, using a radius and availability filter.
US-05	Game Management Module	Develop a game that requires sport, time, place, number of players and ownership by the game organizer.
US-06	Join Request Module	Allow player to join an open game if it is requested with duplicate and capacity check.
US-07	The Organizer Approval Module is a tool used to request approval and acceptance from the organizer.	Join requests are accepted/rejected by organizer and both are informed about the status.
US-08	Use Arena Discovery Module to integrate maps. Use Arena Discovery Module to incorporate maps.	Display nearby arenas by sport, location and slot information.
US-09	Booking Request Module	Make a booking request, check the Pending/Approved/Rejected status and email the manager/admin.
US-10	Recommendation Service - Fallback Ranking	Report games/players to new users when there is no history of the activity.
US-11	The program was a success and will be continued in the future.	Rank suggestions using preferences, distance, activity history and reliability score.
US-12	Rating and moderating feedback. Feedback, Rating & Moderation Module.	Handle post-game feedback once a participant and update the trust signals by using defined rules.
US-13	Notification Service, FCM	Create and send push notifications and in-app notifications for requests, approvals, booking updates and reminders.
US-14	Admin Moderation Module	Reports are reviewed and warnings given by admin, accounts suspended and/or cleared with logged decisions.

3. Detailed System Design

3.1 Module Decomposition

The system is broken down into modules which can be individually implemented and tested. Each module acts as a facade for access through the API layer to the Fire store collections through the data access layer.

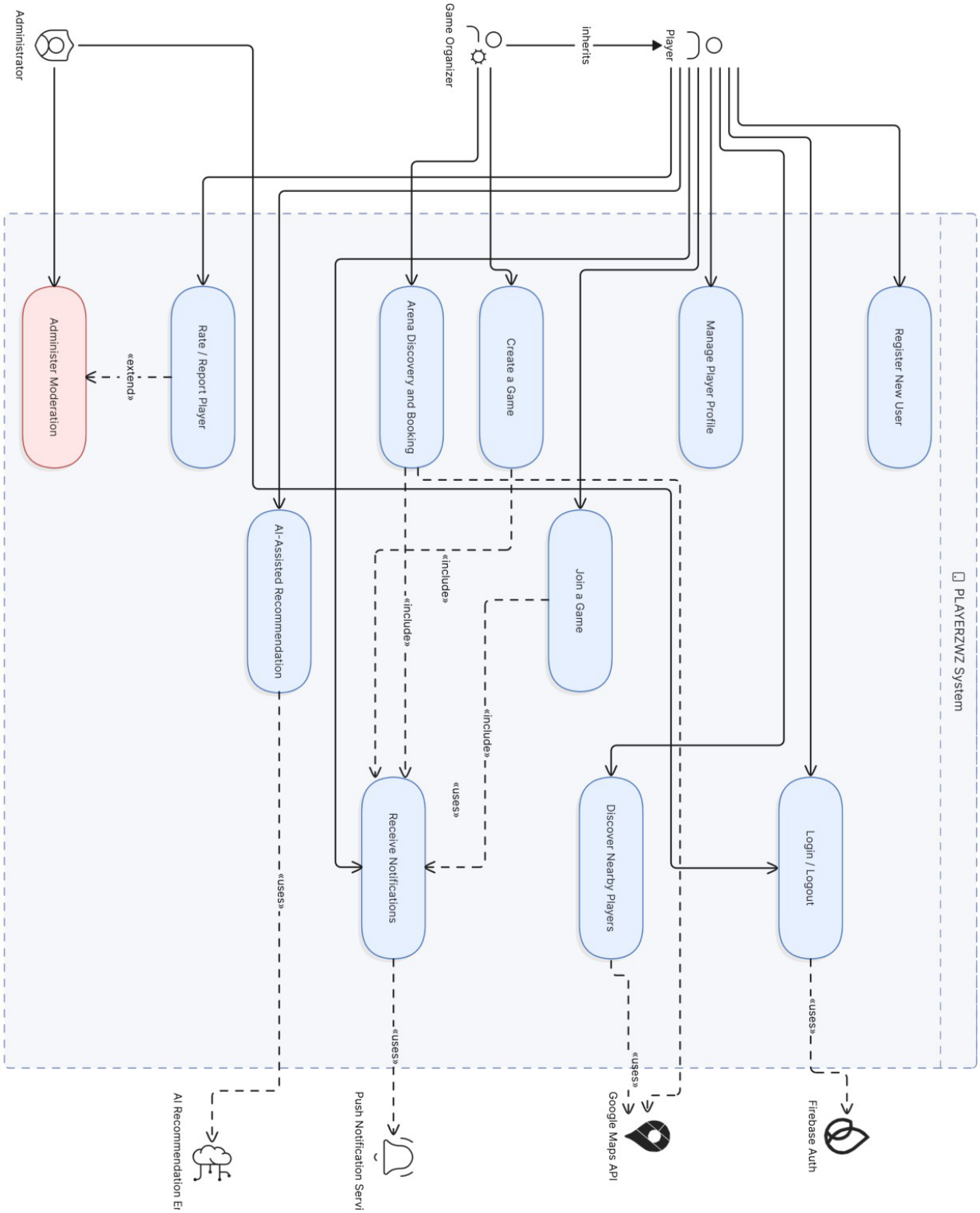
Module	Responsibilities
Authentication Module	Manages register/login/logout, token verification and duplicate email access issues, as well as role-based access.
Profile & Preferences Module	Stores user bio, preferred sports, availability, manual area, radius and reliability score.
Discovery Module	Gets local players and available sport rounds based on sport filters, location/radius and availability.
Game Management Module	Develops games, checks required fields, controls games and keeps track of players.
Join Request Module	Makes pending join requests, doesn't allow duplicate requests, updates participant lists when approved, and checks capacity.
Arena & Booking Module	Accepts booking requests, manages booking status changes, rules for slot conflicts, lists arenas.
Recommendation Module	Computes ranking signals for players, games with fallback and history-based signals.
Feedback & Moderation Module	Allows for the storage of ratings/reports, prevents duplicate rating/reporting and facilitates admin warning/suspension actions.
Notification Module	Records and sends app notifications and FCM for significant asynchronous events.
API Layer	Accepts HTTPS requests, checks payloads, invokes service modules and returns JSON.
Data Access Layer	Aggregates collection names, queries, transactions and error handling in Fire store.

3.2 Data Flow Description

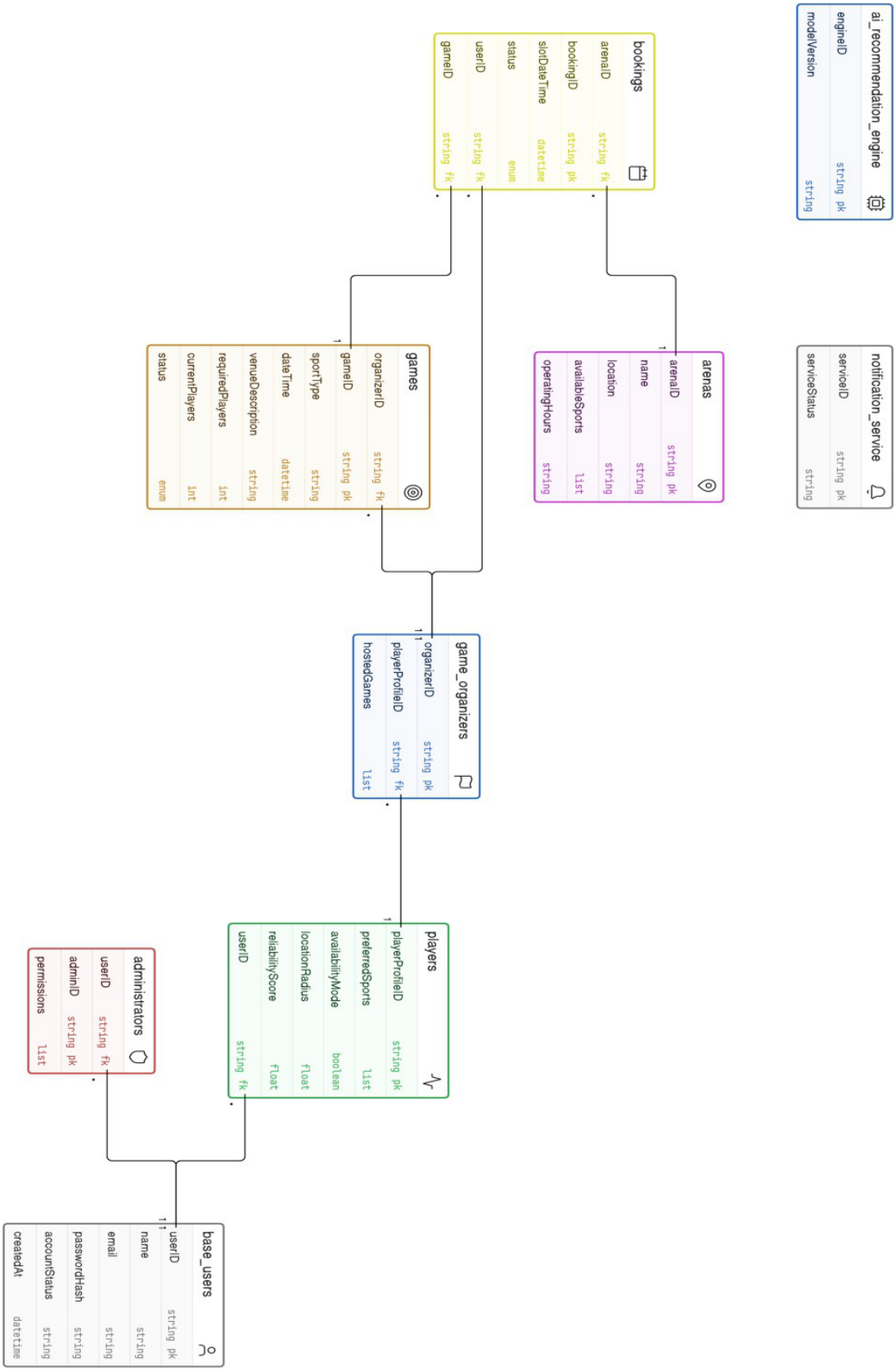
Flow	Data Movement
Registration/Login Flow	User submits data -> Flutter checks for all the required data -> Firebase Authentication creates/signs in account -> API receives token and checks it -> User/profile record is created or retrieved -> Home screen opens.
Nearby Discovery Flow	User chooses sport/radius, app sends current or manually set area to the API, the API returns profiles/games that are compatible, the maps service filters out those that are too far and sorts based on relevance and returns to map/list view.
Create and Join Game Flow	Organizer submits the game form -> API validates sport, time, location, and the number of players required to join the game -> game record is saved, and users close to the organizer can see the game record -> player requests to join the game -> organizer approves/rejects request -> player list and notifications are updated.
Arena Booking Flow	Organizer selects arena slot, API checks for booking duplication/double booking, booking request is stored as Pending, notification is sent to manager/admin, approval/denial will update the booking, game and notification records.
Recommendation Flow	Recommendations are offered by user opening them -> API gets profile, preferences, location, games, history, reliability data -> recommendation service does calculations to return ranked suggestions; if no history is provided, rule-based approaches are used.
Feedback/Report Flow	Game complete -> user submits rating/report -> API verifies the eligibility of the participant and if the feedback has been seen before or not, whatever is selected in the settings file -> the report is saved -> new reliability score is only calculated when a fairness rule is approved by the admin -> admin can view flagged reports.

4. UML Diagrams

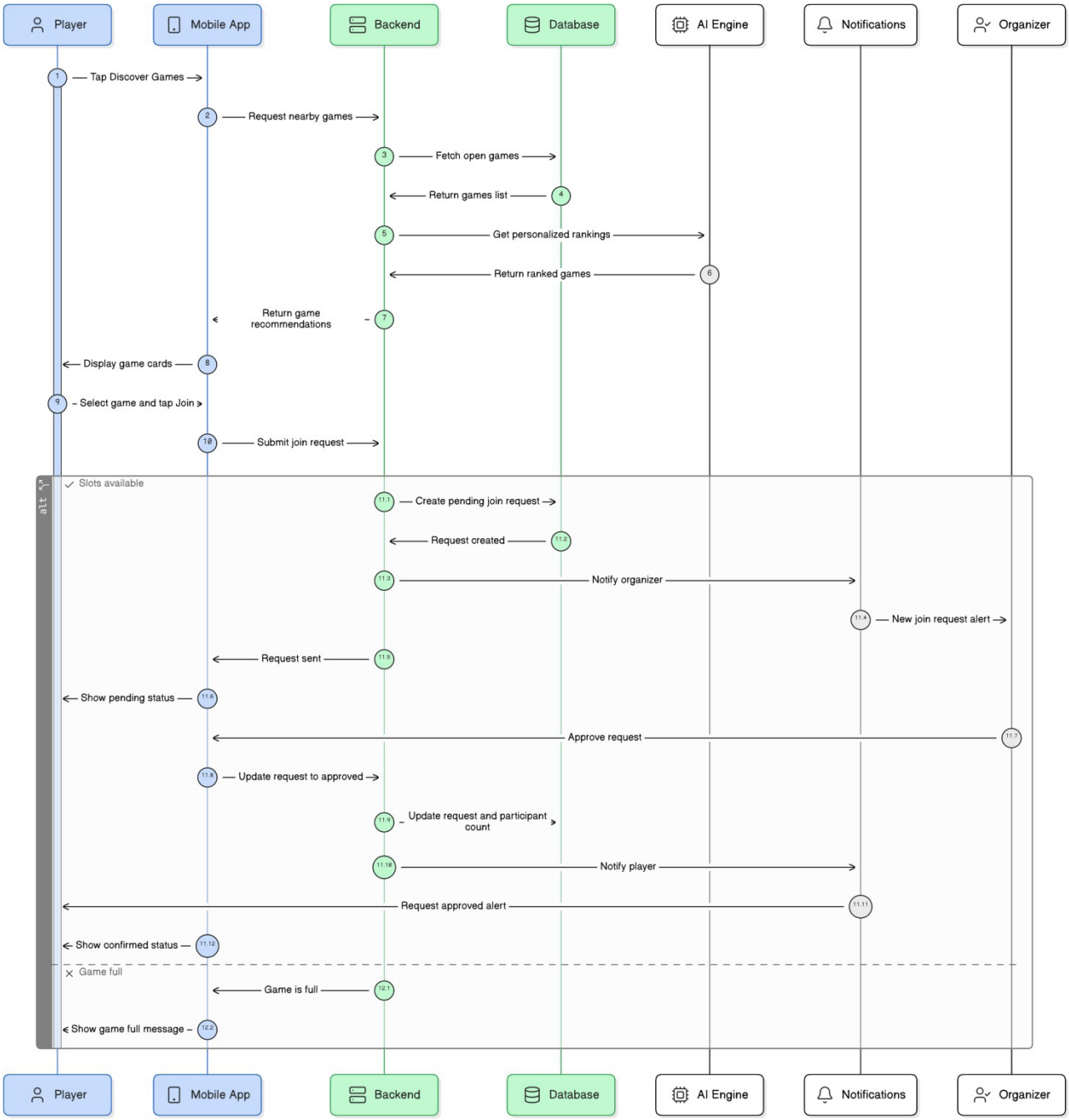
4.1 Use Case Diagram



4.2 Class Diagram

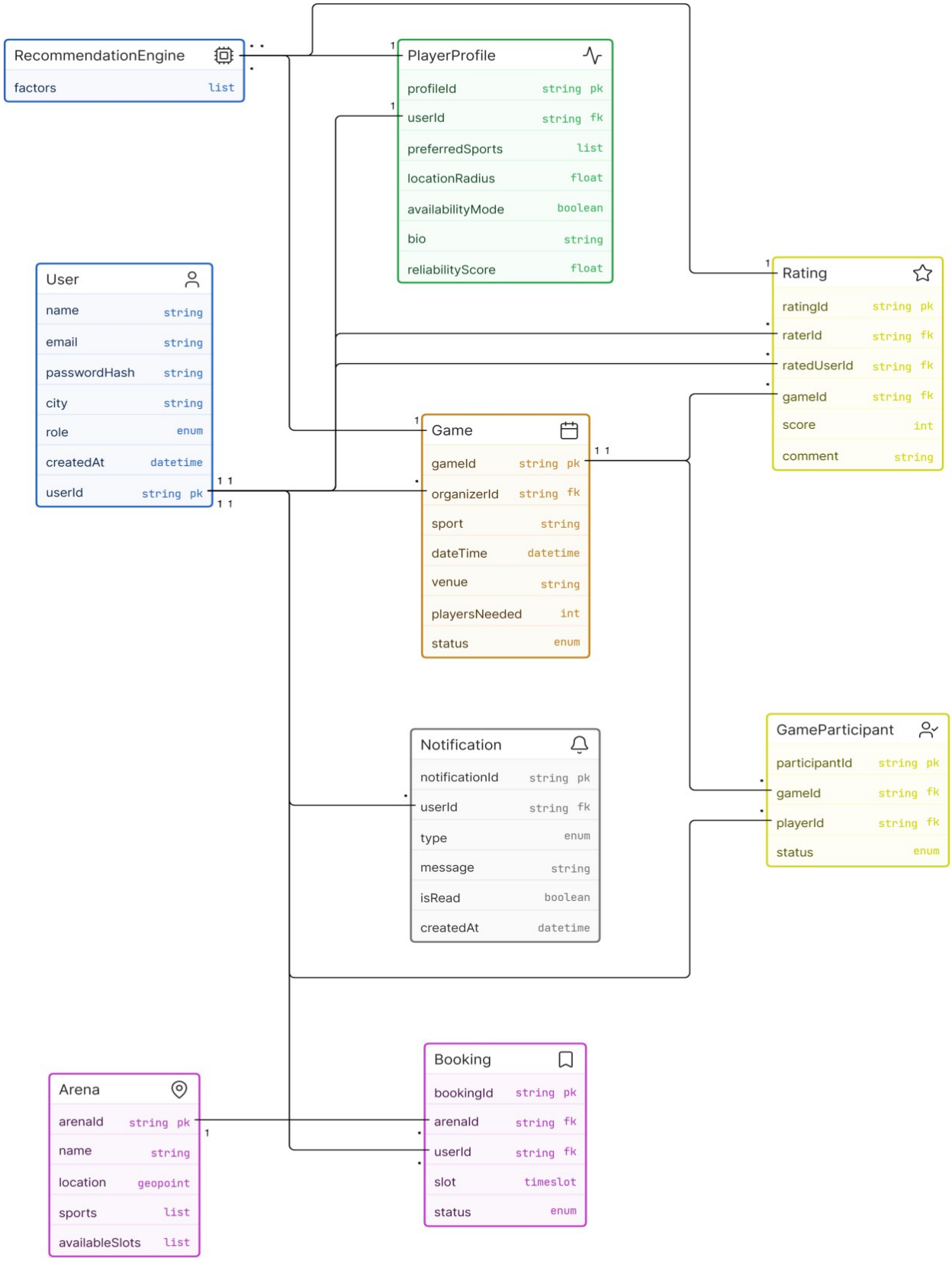


4.3 Sequence Diagrams



5. Database Design (ERD)

5.1 ER Diagram



5.2 Schema Tables

users

Field	Data Type	Description
user_id	String	Primary key / Firebase UID
name	String	Full name
email	String	Unique login email
role	Enum	player, organizer, arena_manager, admin
city	String	User city or manual area
status	Enum	active, warned, suspended
created_at	Timestamp	Record creation time

profiles

Field	Data Type	Description
profile_id	String	Primary key
user_id	String	FK to users
sports	Array<String>	Preferred sports
availability	String	Available, busy or custom mode
radius_km	Number	Discovery radius
bio	String	Optional profile bio
location	GeoPoint	Last known/manual location
reliability_score	Number	Trust score from history

games

Field	Data Type	Description
game_id	String	Primary key
organizer_id	String	FK to users
sport	String	Sport category
start_time	Timestamp	Match date/time
location	GeoPoint/String	Venue coordinates or manual address
required_players	Number	Slots required
status	Enum	open, full, completed, cancelled

game_participants

Field	Data Type	Description
participant_id	String	Primary key
game_id	String	FK to games
user_id	String	FK to users
request_status	Enum	pending, approved, rejected, cancelled
joined_at	Timestamp	Request/join time

arenas

Field	Data Type	Description
arena_id	String	Primary key
manager_id	String	FK to users
name	String	Arena name
sports	Array<String>	Supported sports
address	String	Physical address
location	GeoPoint	Arena coordinates
slot_status	Map/List	Available, pending or booked slots

booking_requests

Field	Data Type	Description
booking_id	String	Primary key
arena_id	String	FK to arenas
game_id	String	Optional FK to games
requested_by	String	Organizer/user id
slot_time	Timestamp	Requested slot
status	Enum	pending, approved, rejected, cancelled
manager_notes	String	Optional response notes

ratings_reports

Field	Data Type	Description
rating_id	String	Primary key
game_id	String	FK to games
from_user_id	String	Reviewer id
to_user_id	String	Reviewed player id
rating	Number	1 to 5 rating
report_reason	String	Optional report/no-show reason
status	Enum	submitted, under_review, resolved

notifications

Field	Data Type	Description
notification_id	String	Primary key
user_id	String	Recipient id
type	String	join, booking, reminder, moderation
message	String	Notification message
is_read	Boolean	Read status
created_at	Timestamp	Created time

admin_actions

Field	Data Type	Description
action_id	String	Primary key
admin_id	String	FK to users
target_user_id	String	User affected
action_type	Enum	warn, suspend, clear
notes	String	Action explanation
created_at	Timestamp	Action time

6. API Design

The API design specifies the contracts between the Flutter front end and backend services. The authentication process is done using Firebase, and the Firebase token is validated on the API using protected operations.

Endpoint	Method	Input	Output
/api/auth/register	POST	name, email, password, city, favorite_sport	user_id, profile_id, token/status
/api/auth/login	POST	email, password	token, user role
/api/profile/{userId}	GET	Firebase token	profile data, preferences, reliability score
/api/profile/{userId}	PUT	sports, bio, availability, radius, location	updated profile
/api/discovery/players	GET	sport, radius_km, location/manual_area	compatible players list/map data
/api/games	POST	sport, date_time, location, required_players	created game object
/api/games/open	GET	sport, radius_km, location	open games list
/api/games/{gameId}/join	POST	player_id/token	join_request_id, pending/accepted status
/api/join-requests/{requestId}	PATCH	approve/reject action, organizer token	updated request and participant list
/api/arenas	GET	sport, location, radius	arena list with slots
/api/bookings	POST	arena_id, game_id, slot_time, notes	booking request as Pending
/api/bookings/{bookingId}/status	PATCH	approved/rejected, manager notes	updated booking and notifications
/api/recommendations	GET	user token, location/sport filters	ranked games and players
/api/ratings	POST	game_id, to_user_id, rating/report	saved feedback and trust update
/api/admin/reports	GET	admin token	pending reports and affected users
/api/admin/actions	POST	report_id, action_type, notes	logged moderation decision
/api/notifications	GET	user token	in-app notification list

7. UI/UX Design (Prototypes)

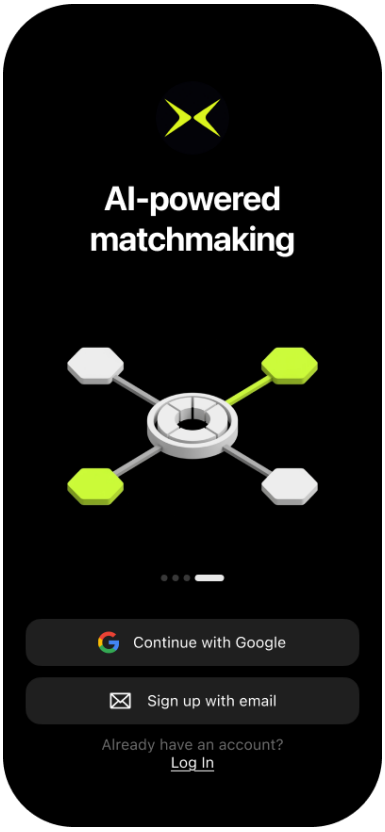
Simple user interface design flow, mobile first. Important actions that were not completed are still shown on the home and maps and lists support the location based and manual area users.

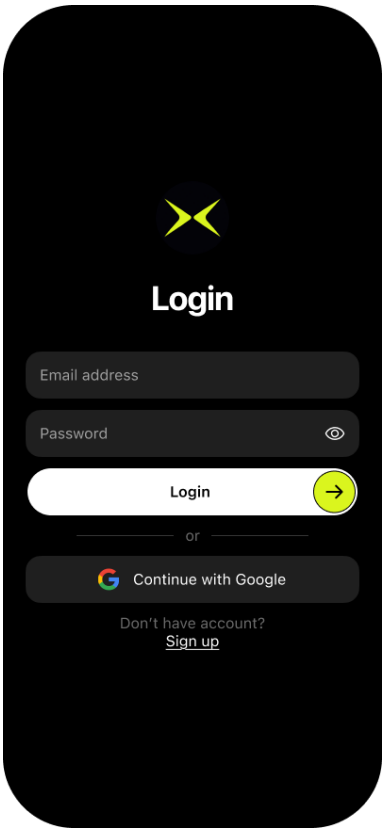


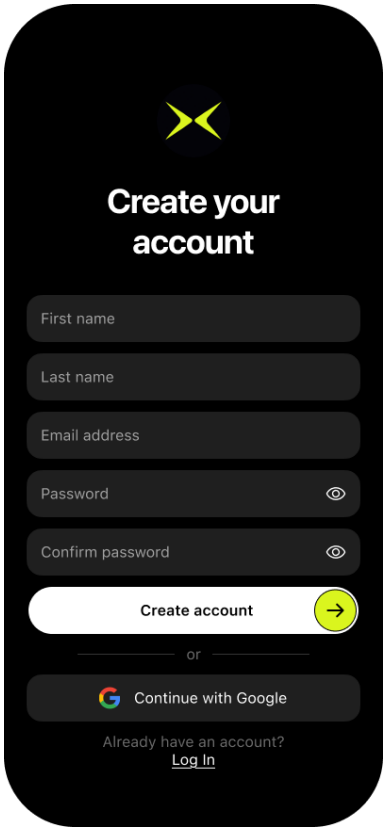


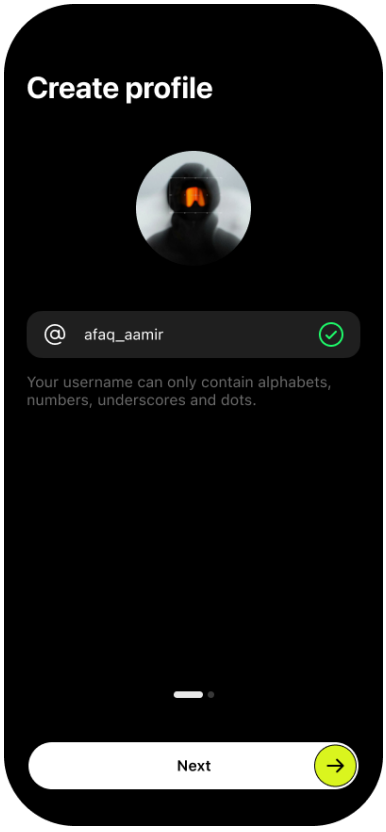


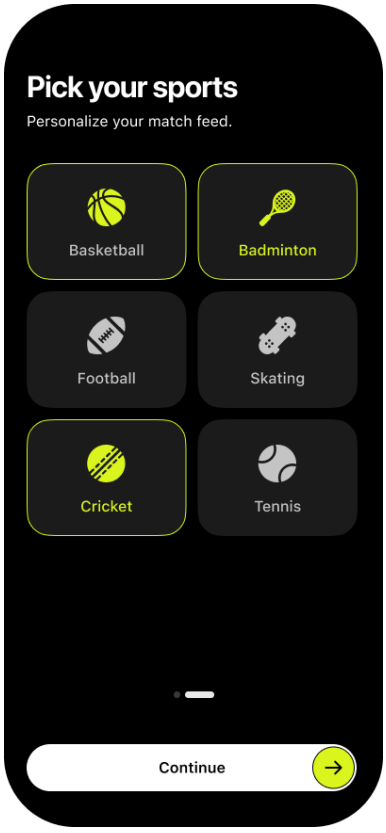

Find nearby players
Continue with Google
Sign up with email
Already have an account?
Log In

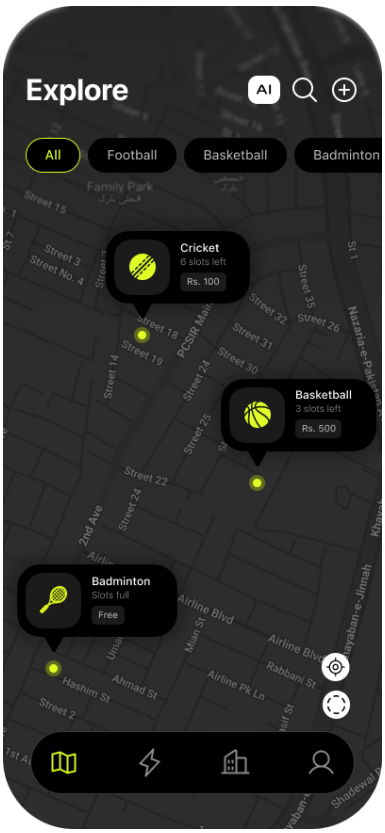

AI-powered matchmaking
Continue with Google
Sign up with email
Already have an account?
Log In

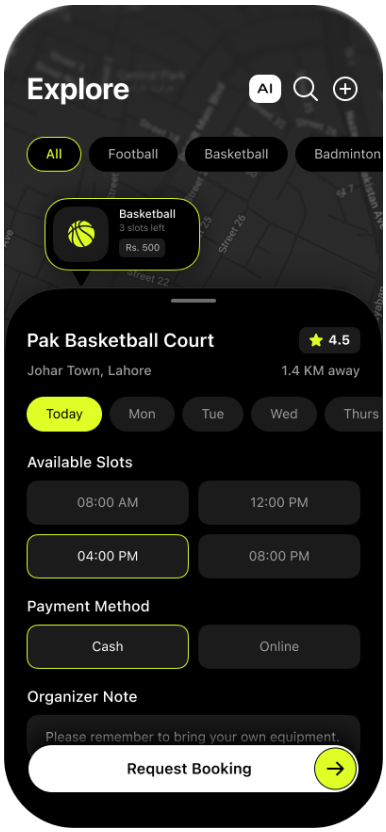

Login
Email address
Password
Login
or
Continue with Google
Don't have account?
Sign up

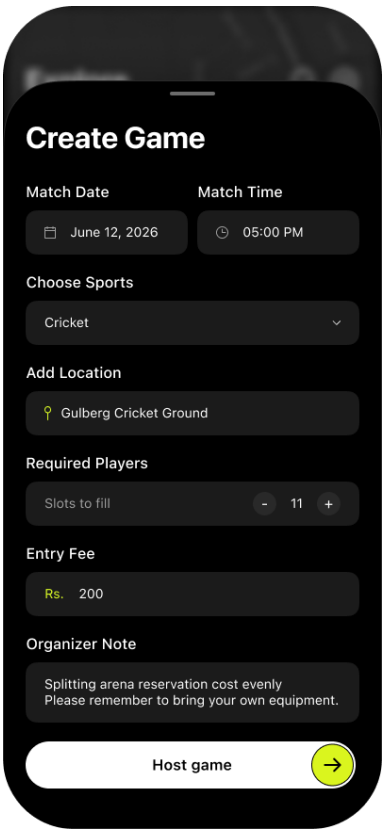

Create your account
First name
Last name
Email address
Password
Confirm password
Create account
or
Continue with Google
Already have an account?
Log In

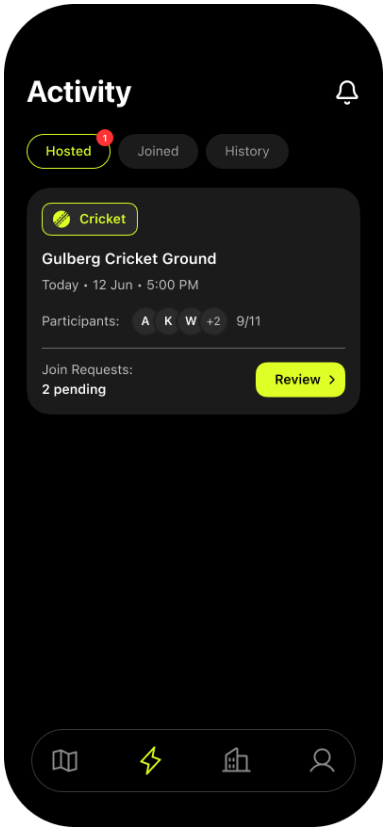

Create profile
afaq_aamir
Your username can only contain alphabets, numbers, underscores and dots.
Next

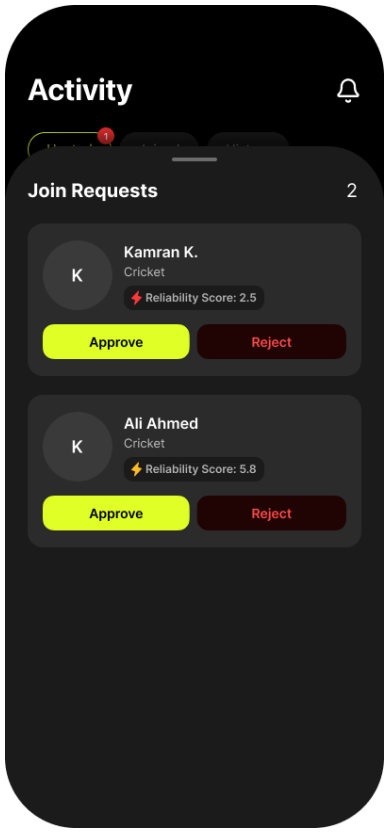

Pick your sports
Personalize your match feed.
Basketball
Badminton
Football
Skating
Cricket
Tennis
Continue

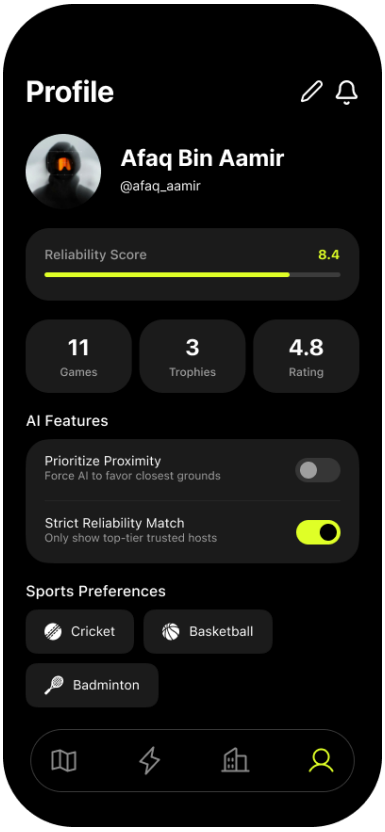

Explore
AI
All
Football
Basketball
Badminton
Cricket
6 slots left
Rs. 100
Basketball
3 slots left
Rs. 500
Badminton
Slots full
Free


Explore
AI
All
Football
Basketball
Badminton
Basketball
3 slots left
Rs. 500
Pak Basketball Court
4.5
Johar Town, Lahore
1.4 KM away
Today
Mon
Tue
Wed
Thurs
Available Slots
08:00 AM
12:00 PM
04:00 PM
08:00 PM
Payment Method
Cash
Online
Organizer Note
Please remember to bring your own equipment.
Request Booking

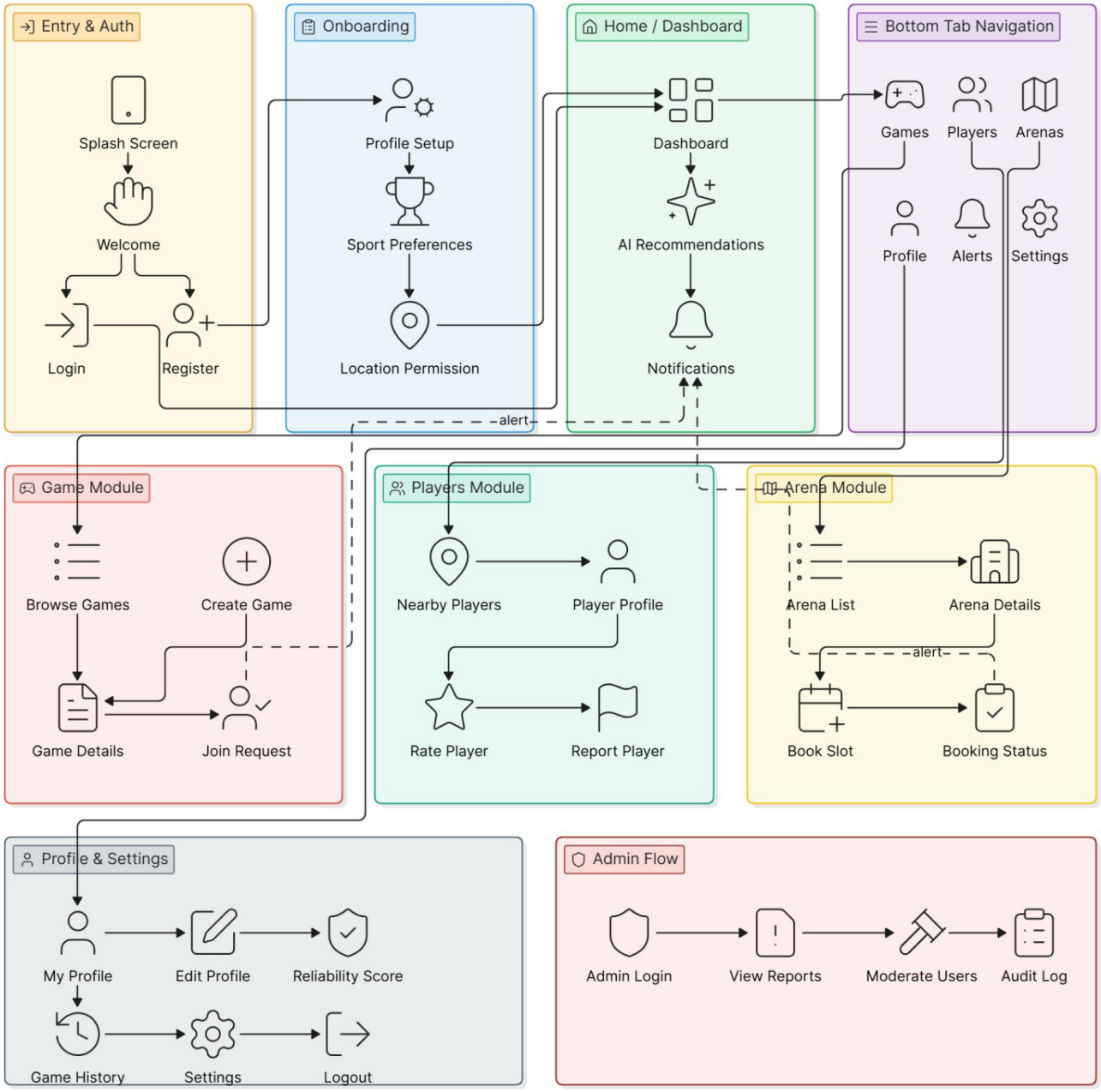

Create Game
Match Date
June 12, 2026
Match Time
05:00 PM
Choose Sports
Cricket
Add Location
Gulberg Cricket Ground
Required Players
Slots to fill
11
Entry Fee
Rs. 200
Organizer Note
Splitting arena reservation cost evenly
Please remember to bring your own equipment.
Host game


Activity
Hosted
Joined
History
Cricket
Gulberg Cricket Ground
Today • 12 Jun • 5:00 PM
Participants:
9/11
Join Requests:
2 pending
Review


Activity
Join Requests
2
Kamran K.
Cricket
Reliability Score: 2.5
Approve
Reject
Ali Ahmed
Cricket
Reliability Score: 5.8
Approve
Reject


Profile
Afaq Bin Aamir
@afaq_aamir
Reliability Score
8.4
11
Games
3
Trophies
4.8
Rating
AI Features
Prioritize Proximity
Force AI to favor closest grounds
Strict Reliability Match
Only show top-tier trusted hosts
Sports Preferences
Cricket
Basketball
Badminton

7.1 Navigation Flow



7.2 Screens and Prototype Notes

Screen	Prototype / Wireframe Description
Splash / Login / Register	Email/password fields, city, favorite sport and easy validation.
Profile & Preferences	Sports list, availability mode, radius slider, bio and manual/current location option.
Nearby Discovery	Map/list toggle with sport/distance filters and compatible player toggles.
Create Game	Form for sport, time, venue/manual location, players and organizer notes.
Game Details	Information about the game, how many are playing, the join button, and the status of the game's organizer.
Arena List & Booking	The Arenas are divided by sport/location, with the slot information and request booking button.
Recommendations	Add explanation tags to ranked games/players, nearby, same sport, reliable etc.
Feedback / Report	Options for post-game rating and reporting, including duplicate submission prevention.
Admin Moderation	Record list, target student information, warning/suspension/clear action and audit notes.

7.3 Wireframe Rules

- Primary action buttons should be in the same location on all main screens.
- The user should not have to seek technical assistance to register, create a game or request to join a game.
- Location permission denial shouldn't prevent the entire app; manual area entry should still be available.
- Filters for sport, distance and time should be easy to find and use on discovery screens and should be displayed at the top of the screen.
- The error states must be informative, such as Invalid Credentials, Slot Unavailable or Request Already Sent.

8. Sprint-wise Design Evolution

The design for Phase-II is identical to the Phase-I sprint plan, but explains how each sprint affects the architecture, database and API decisions.

Sprint	Features Designed	Design Improvements
Sprint 1	Authentication, base project structure, profile screens and database collections.	Users/profiles collections separated to maintain security and preferences modular, Firebase Auth selected.
Sprint 2	Discovery, game creation, join request and approval by an organizer flow.	The capacity and request states are maintained consistent as game and participant records are separated.
Sprint 3	Listing and booking for Arena, notifications and status updates.	Pending/Approved/Rejected status is used in booking requests to prevent displaying bogus confirmed slots.
Sprint 4	Recommended for services such as recommendation logic, rating/report flow, testing and documentation, and admin moderation.	Service to calculate recommendations separated from UI and API to enable better AI ranking system in the future.

9. Test Design

Test design aims at validating critical flows, edge cases and non-functional expectations at academic prototype scale.

Test ID	Scenario	Expected Result
TD-01	New user signs up with valid e-mail, password, city and favorite sport.	Account/Profile is made.
TD-02	The user types in an email address or incorrect login password.	Readable error comes and App does not crash.
TD-03	User adds sports, availability and radius.	The discovery/recommendation uses the new values and profile is saved.
TD-04	User refuses to grant location access, and manually enters area.	Discovery is still working with manual area or selected city.
TD-05	Organizer makes a game up with a valid sport, time and location.	The game can be saved and is open to other users in the vicinity who are eligible to use it.
TD-06	Duplicate join to same game by Player.	System does not allow duplicate request to be made and displays the already requested message.
TD-07	Organizer gives approval if capacity is available.	The list of participants is updated and the player is notified of his/her approval.
TD-08	Request for booking will be made for a pending/approved booking.	Conflict is identified and no double bookings.
TD-09	When user is new, they are presented with a screen of recommendations with no history.	Fallback suggestions are shown based on the rules.
TD-10	Regular user opens recommendation screen, with history.	Results are displayed by order of preference, distance, activity and reliability.
TD-11	Player enters rating/report twice for the same player/ game.	One feedback record will be accepted.
TD-12	Admin warns/suspends reported user.	User status changes are appropriately recorded and user status is accurate.
TD-13	Network interruption is when the network stops working during request.	App displays meaningful error message and doesn't destroy local state.
TD-14	We have 50 academic test users that will be accessing the core flows at the same time.	No noticeable slowdown in prototype test environment.

10. Design Quality Attributes

Attribute	Design Handling
Scalability	Later versions will include more users, arenas and sports with the use of Fire store collections, geohash/radius filters and separate service modules. Discovery and Recommendation must be done using indexes and paginated queries for prototype scale.
Security	Passwords are not stored in the database. The role checks, API calls need verified tokens and the credential management is done by Firebase Auth. Sensitive operations such as moderation and booking status updates are logged.
Maintainability	The UI, API, services, data access and recommendation logic are separated. The responsibilities are clearly defined for each module and the modules can be tested independently.
Reliability	Explicit states are used for join requests, booking status, ratings and reliability updates. Duplicate requests, duplicate feedback and inconsistent booking states are handled through validation and transaction-like updates.
Performance	Proximity search should return the first results within 5 seconds of a specified distance. For prototype data, the recommendation fallback should occur within three seconds and the history-based ranking should occur within six seconds.
Usability	Simple forms, readable labels, map/list views, clear error messages are used in main workflows. When location permission is denied, there is a manual area entry option.
Fairness & Ethics	Reliability score is only altered through defined rules. No show cases and reports should be reviewed in advance of the imposition of severe penalties.

11. Deployment Considerations

The academic prototype can be deployed by an easy-to-demonstrate and maintain practical cloud stack.

Area	Deployment Decision
Mobile App	Android type Flutter build to test on emulators and physical devices. Support of iOS can be rolled out in the future with the same flutter code.
Backend API	Hosted on Flask REST API on Render, Heroku, AWS Elastic Beanstalk or Google Cloud Run (as available at university / demo).
Authentication	Email/password sign-in and verifying a token with Firebase Authentication in API middleware.
Database	Cloud Fire store collections for User, Profile, Game, Participants, Arenas, Bookings, Ratings, Notifications and Admin actions.
Maps & Location	Usage limit documented and fallback manual entry of area through Google Maps SDK and Places/Geocoding API.
Notifications	Push notifications with Firebase Cloud Messaging and in-app logs with Fire store notifications collection.
Version Control	Using a GitHub repository including main, develop and feature branches. PRs are considered before being merged to develop/main.
Configuration	API Keys & Firebase service information should not be embedded in source code but stored in environment variables or secure config files.

12. Future Enhancements

- Recommend more with better machine learning recommendations based on real user participation history.
- Add authenticated Arena Manager portal and Calendar based slot management.
- Include options to create tournaments and schedule matches in a league format with a future update.
- Include payment integration for arena booking only once validated in academic prototype.
- Include Social Login, player badges and team groups, but retain privacy options.
- Integrate a new process for resolving unfair ratings and no-show reports.
- Extend mobile support to mobile iOS and web admin dashboards following core Android prototype stability.

13. Revised Project Plan

The updated project plan begins with the baseline of completed Phase-I requirements and unfolds to the end of Phase-II design finalization and into prototype implementation sprints. The plan prioritizes important stories over optional commercials.

Milestone	Status	Notes
Phase-I SRS Completed	Done	Requirements, backlog, user stories, non-functional requirements and risk analysis prepared.
Phase-II SDS Architecture	Done	Architecture, components and data flow prepared, and mapping of designs.
UML and Database Design	Done	Use case, class, sequence diagrams, ERD and schema tables prepared.
API and UI/UX Design	Done	Endpoints, navigation flow, screen descriptions and prototype rules prepared.
Testing and IV&V	In Progress	Test scenarios and design defects now logged, implementation defects will be added later.
Prototype Sprint 1	Planned	Implementing authentication, profile and base database.
Prototype Sprint 2	Planned	Discovery, game management, booking, recommendations and moderating flows.


Phase / Task
Semester 1 (M1 - M6)
Semester 2 (M7 - M12)
M1
M2
M3
M4
M5
M6
M7
M8
M9
M10
M11
M12
Phase 1: Requirements & Planning
Requirement analysis & feasibility
User needs survey & observations
Finalize system architecture
Tool & technology selection
Phase 2: Design & Development
Flask backend development
Firebase Auth & Firestore setup
Profile, game & matchmaking APIs
Location-based logic
Flutter mobile app development
Registration, login & profile screens
Game creation & joining interfaces
Google Maps integration
Phase 3: AI Module & Integration
AI-assisted matchmaking logic
Rule-based + ML hybrid system
Player rating & reliability scoring
Availability mode & smart notifications
Phase 4: Testing, Optimization & Submission
No-show & misuse detection logic
AI recommendation testing
Performance optimization & bug fixing
User experience improvements
Final system testing & validation
Deployment preparation
Documentation & final report
Presentation, demo & viva prep
Legend - Project Phases
Phase 1: Planning
Phase 2: Design & Development
Phase 3: AI Module & Integration
Phase 4: Testing & Submission

14. References

- P. Gupta and A. Pathak, "Consumer behavior and decision-making in online platforms," International Journal of Information Systems, 2020.

- M. Rahman, M. Uddin, and R. Alam, "Location-based recommendation systems for real-time applications," Journal of Mobile Computing, 2021.

- R. Saha, D. Kumar, and A. Mehta, "Machine learning approaches for user recommendation systems," IEEE Access, 2022.

- L. Chen and Y. Wang, "Trust and reputation mechanisms in online communities," ACM Computing Surveys, 2019.

- Google Developers, "Firebase Authentication and Cloud Firestore," Google Developers Documentation, 2024. Available: https://firebase.google.com/docs

- Flutter Team, "Flutter Framework Documentation," 2024. Available: https://docs.flutter.dev

- Scikit-learn Developers, "Machine Learning in Python Documentation," 2024. Available: https://scikit-learn.org/stable/documentation.html

- Google Maps Platform, "Maps SDK and Places API Documentation," 2024. Available: https://developers.google.com/maps

Appendix A: Glossary

Term	Definition
AI Recommendation Engine	A module to rank the games and players in terms of their preference of sport, length of distance, availability, historical score and reliability.
Arena	A sports ground or futsal court or a badminton court (registered).
Availability Mode	A profile setting to show a player if he/she is available to play games.
Booking	Request for the Arena to be used on a given date and at a specific time for a specific purpose.
Cloud Fire store	Implemented Firebase NoSQL database for storing Users, profiles, games, bookings, rating and notifications.
Coordination Friction	Lots of wasted time and confusion from manually creating sports games through calls and group chats.
Game Organizer	The user who creates a game, manages requests from participants and can request an arena booking.
Geohash	Location indexing technique that allows to efficiently locate near users or near records.
Join Request	Request from a player to play an open game.
No-show	A player that has confirmed and has not shown up to a game without appropriate cancelling.
Player	Main users of a mobile application who discover games, join games, edit profile and get recommendations.
Reliability Score	Attendance, ratings, reports and no show history based trust score.
Scrum	An agile approach to work organization in terms of sprints, reviews and continuous improvement.
Sprint	A fixed development cycle of about 2 weeks and a selected backlog item.
Use Case	A clear "snapshot" of the pattern of behavior(s) a person uses in the system to meet a goal.

Appendix B: IV & V Report

(Independent Verification & Validation)

IV & V Resource

Name: Afaq Ul Islam Signature: _____

S#	Defect Description	Origin Stage	Status	Fix Time
1	Although the information around notification and external map services was present in the architecture diagram initially, it was not clearly expressed.	SDS Design Review	Resolved	0 h 35 m
2	There was a need for conflict handling for pending or already approved slots in booking workflow.	Database/API Review	Resolved	0 h 50 m
3	Provide an alternative for users who have no history: Recommendation design needed fallback path for users with no history.	Algorithm Review	Resolved	0 h 40 m
4	Class diagram made a distinction between Game, Join Request and Booking Request.	UML Review	Resolved	0 h 45 m
5	Feedback cases and join request cases to be duplicated explicitly needed to be built in to the test design.	Test Design Review	Resolved	0 h 30 m

Total Defects Found	Resolved	Pending	Total Fix Time
5	5	0	3 hours 20 minutes

IV & V Notes

Design Defects listed were reviewed at the time of internal checks before this SDS was completed. Implementation-related IV&V defects, validation results and fixes should be added at the prototype development and final prototype testing stages.